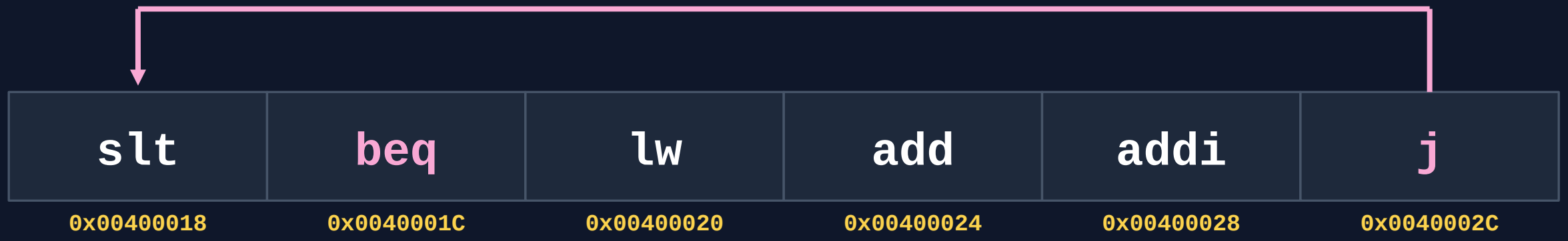
ENCONTRO 4 DE 6

Controle de fluxo

Curso Introdutório de Programação de Baixo Nível



Dois problemas ficaram sem solução

Repetição

```
lw   $t1, 4($s0)
add  $t2, $t2, $t1
lw   $t1, 8($s0)
add  $t2, $t2, $t1
…
```

Cinco elementos, cinco pares copiados. Com mil elementos, mil pares?

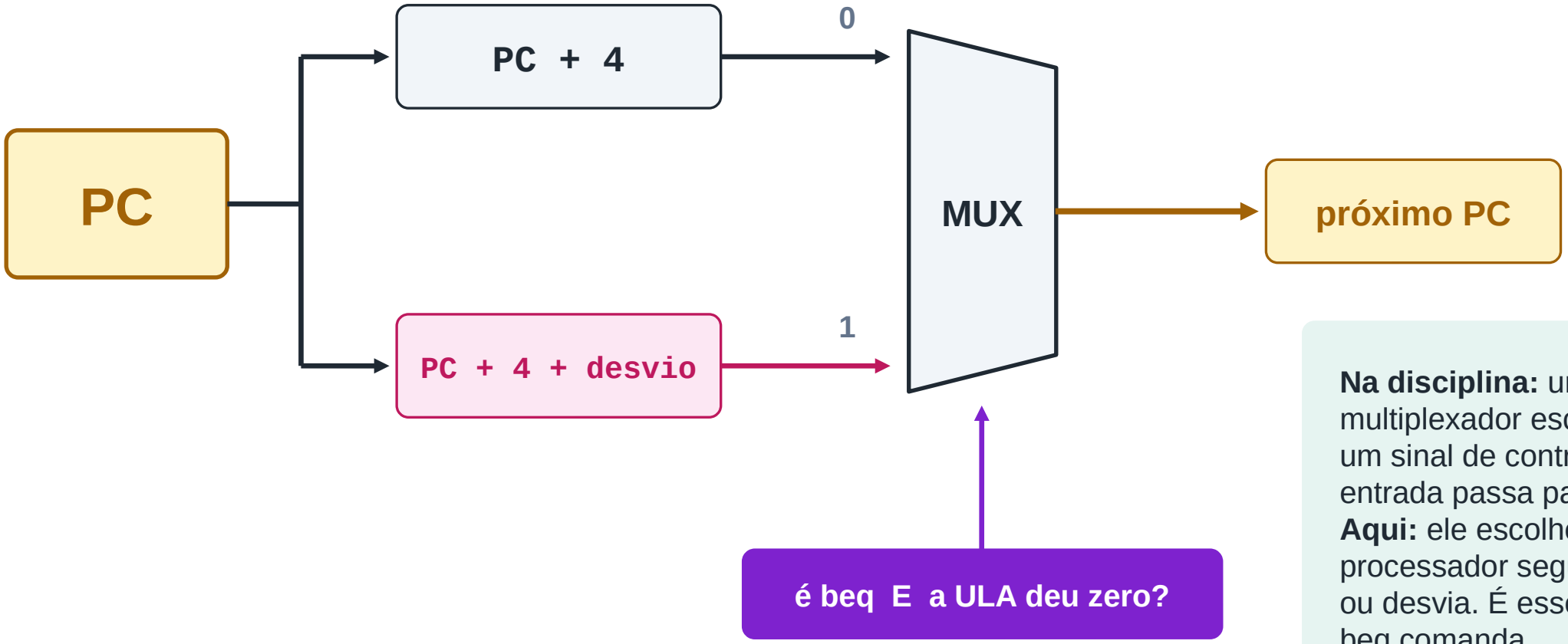
Decisão

```
maior = vetor[0]
se vetor[i] > maior:
    maior = vetor[i]
```

Qual é o maior elemento? É preciso comparar — e fazer coisas diferentes conforme a resposta.

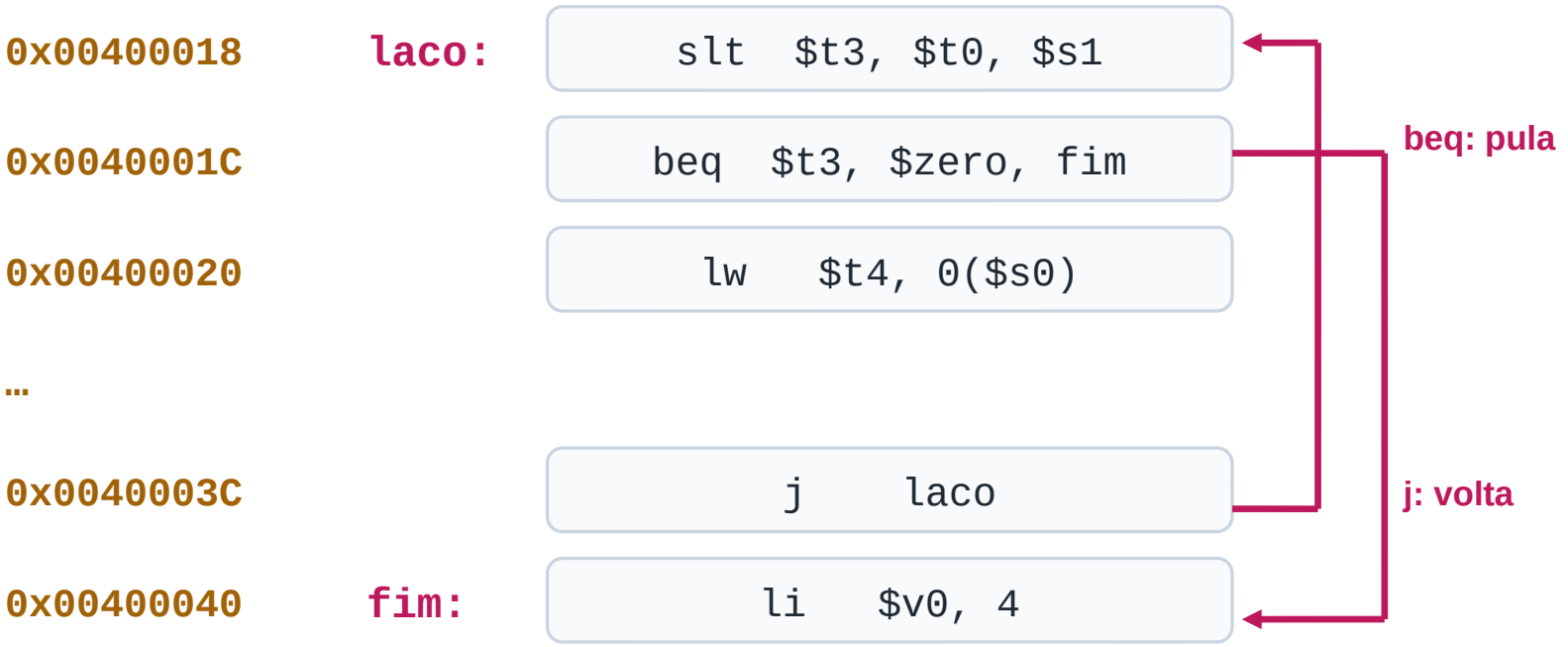
Até hoje, o processador só andou para a frente: uma instrução depois da outra, de 4 em 4 bytes. Hoje ele aprende a voltar, a pular — e a decidir quando fazer cada coisa.

Um multiplexador escolhe qual é a próxima instrução



Na disciplina: um multiplexador escolhe, com um sinal de controle, qual entrada passa para a saída. **Aqui:** ele escolhe se o processador segue em frente ou desvia. É esse fio que o beq comanda.

Desviar é escrever outro endereço no PC



Sem desvio: PC ← PC + 4. Foi assim até hoje.

Com desvio: PC ← o endereço do rótulo. A próxima instrução é outra.

O rótulo vira endereço, como main: e vetor: nas aulas anteriores. laco é só um nome para 0x00400018.

j: vá para lá, sempre

`j  laco`

PC ← endereço de laco

```
laco:
    li   $v0, 1
    li   $a0, 7
    syscall
    j    laco
```



Incondicional: não testa nada. Sempre desvia.

Sozinho, faz um laço infinito: este programa imprime 7777777… até alguém pará-lo.

Para sair de um laço, é preciso uma instrução que às vezes desvie e às vezes não.

Desvio condicional: só pula se…

`beq $t0, $t1, L`

branch if equal

se $t0 = $t1, vá para L
senão, segue para a próxima

`bne $t0, $t1, L`

branch if not equal

se $t0 ≠ $t1, vá para L
senão, segue para a próxima

Só comparam igualdade. Para “menor que”, falta uma peça. E um truque útil: comparar com **`$zero`** — beq $t3, $zero, L quer dizer “se $t3 vale 0, vá para L”.

Em C: if (a == b) e if (a != b) — os dois casos mais simples de decisão.

slt compara e responde 1 ou 0

`slt  $t3, $t0, $s1`

`$t3 ← 1 se $t0 < $s1;  senão, $t3 ← 0`

$t0	$s1	$t3
3	8	**1**
8	8	**0**
9	8	**0**
−2	8	**1**

Para “maior que”, troque a ordem: a > b é o mesmo que b < a.

Com slt + beq/bne, toda comparação vira duas linhas:

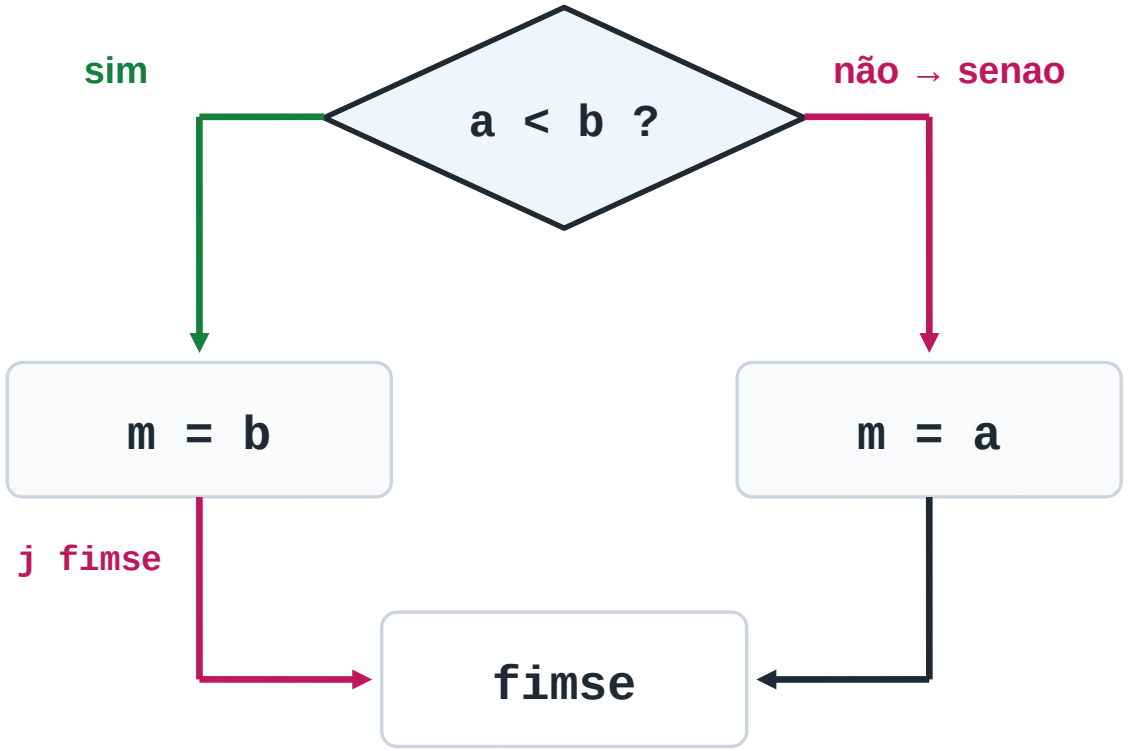
Vá para L se…	Escreva
a < b	`slt $t, a, b` `bne $t, $zero, L`
a ≥ b	`slt $t, a, b` `beq $t, $zero, L`
a > b	`slt $t, b, a` `bne $t, $zero, L`
a ≤ b	`slt $t, b, a` `beq $t, $zero, L`

O MARS aceita blt, bge, bgt e ble: são pseudoinstruções que viram exatamente estes pares, com $at como $t. No curso, escrevemos o par à mão.

if / else: o maior de dois números

```
if (a < b)
    m = b;
else
    m = a;
```

```
        slt  $t0, $s0, $s1  # a < b ?
        beq  $t0, $zero, senao
        move $s2, $s1       # m = b
        j    fimse
senao:  move $s2, $s0       # m = a
fimse:
```



O j no fim do “então” é obrigatório: sem ele, o processador cairia direto no “senão”.

Em assembly, a condição aparece invertida

C diz

faça o bloco SE a condição for verdadeira

```
if (a < b) { … }
```

Assembly diz

PULE o bloco SE a condição for falsa

```
slt $t0, a, b
beq $t0, $zero, depois
```

Por quê? O desvio serve para sair do caminho. Se a condição é verdadeira, o processador simplesmente segue em frente e entra no bloco. Só quando ela é falsa é preciso pular.

Regra prática: escreva o desvio com a condição que faz PULAR o bloco.

while: o teste no topo, o j no fim

```
x = 1;
while (x < 100)
    x = x + x;
```

```
      li   $t0, 1
      li   $t5, 100
laco: slt  $t2, $t0, $t5
      beq  $t2, $zero, fim
      add  $t0, $t0, $t0
      j    laco
fim:
```



1 **teste** x < 100? Se não, sai

2 **corpo** x = x + x

3 **volta** j laco: testar de novo

volta	1	2	3	4	5	6	7
x	2	4	8	16	32	64	128

Na oitava vez, 128 < 100 é falso: o beq desvia para fim.

for é um while com as partes arrumadas

```
for (i = 0; i < n; i++) {
    soma = soma + vetor[i];
}
```

inicialização — roda uma vez, antes de tudo

teste — no topo de cada volta; se falso, sai

corpo — o trabalho de cada volta

incremento — no fim de cada volta, antes do j

```
        li   $t0, 0
laco:   slt  $t3, $t0, $s1
        beq  $t3, $zero, fim
        …    corpo    …
        addi $t0, $t0, 1
        j    laco
fim:
```

Toda tradução de for segue este molde. Muda o corpo; o esqueleto é sempre o mesmo.

Um registrador que anda de 4 em 4

vetor[i] está em base + 4 · i. Num laço, há dois jeitos de chegar lá:

Calcular 4 · i a cada volta

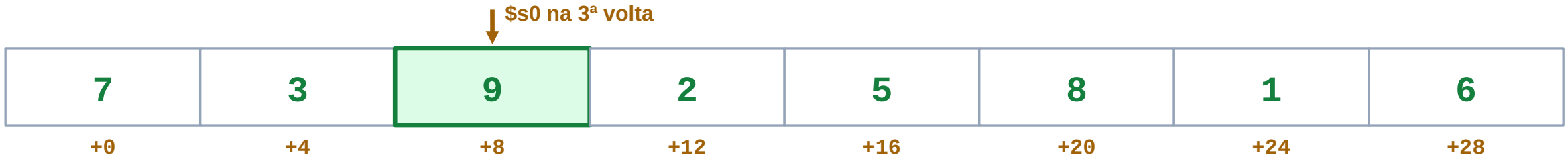
```
add  $t5, $t0, $t0   # 2i
add  $t5, $t5, $t5   # 4i
add  $t5, $t5, $s0   # base+4i
lw   $t4, 0($t5)
```

Funciona, mas gasta três instruções por volta.

Andar com o endereço

```
lw   $t4, 0($s0)
…
addi $s0, $s0, 4     # próxima palavra
```

Uma instrução por volta. É o que o programa da aula faz — e o que os compiladores costumam fazer.



Soma e maior, num laço só

```
        la   $s0, vetor
        li   $s1, 8           # n
        li   $t0, 0           # i
        li   $t1, 0           # soma
        lw   $t2, 0($s0)      # maior
laco:   slt  $t3, $t0, $s1
        beq  $t3, $zero, fim
        lw   $t4, 0($s0)
        add  $t1, $t1, $t4
        slt  $t3, $t2, $t4
        beq  $t3, $zero, pula
        move $t2, $t4
pula:   addi $t0, $t0, 1
        addi $s0, $s0, 4
        j    laco
fim:    … imprime …
```

- inicialização
- teste do for
- corpo (com um if dentro)
- incremento

Um if dentro de um for: o segundo par slt + beq decide se o maior muda. Se vetor[i] não for maior, pula o move.

Run I/O

```
Soma = 41
Maior = 9
```

O estado no fim de cada volta

volta	i	vetor[i]	vetor[i] > maior?	soma	maior
1	0	**7**	não → pula	7	7
2	1	**3**	não → pula	10	7
3	2	**9**	sim → move	19	9
4	3	**2**	não → pula	21	9
5	4	**5**	não → pula	26	9
6	5	**8**	não → pula	34	9
7	6	**1**	não → pula	35	9
8	7	**6**	não → pula	41	9

Depois da 8ª volta, i vale 8; o teste 8 < 8 dá falso, e o beq desvia para fim.

Na 1ª volta, 7 > 7 é falso: o maior já começou valendo vetor[0].

Na atividade, vocês vão montar esta tabela no MARS, com um ponto de parada.

Como o destino cabe na instrução

`beq  $t3, $zero, fim  →  0x11600008`



op	rs	rt	imediato
`000100`	`01011`	`00000`	`0000000000001000`
4 · beq	$t3	$zero	8 instruções adiante de PC + 4

Formato I. O imediato conta instruções a partir da seguinte: 0x0040001C + 4 + 8 × 4 = 0x00400040 = fim. Negativo, volta.

`j  laco  →  0x08100006`



op	endereço (26 bits)
`000010`	`00 0001 0000 0000 0000 0000 0110`
2 · j	0x0100006 × 4 = 0x00400018 = laco

A pergunta do encontro 2: como 26 bits indicam um endereço de 32? Todo endereço de instrução é múltiplo de 4 — os 2 bits de baixo são sempre 00 e nem são guardados. Os 4 de cima vêm do próprio PC.

Traduzam comigo: valor absoluto

```
if (x < 0)
    x = 0 - x;
```

```
      slt  $t0, $s0, $zero
      ____ $t0, $zero, fim
      sub  $s0, $zero, $s0
fim:
```

Lembrem do truque: o desvio é para PULAR o bloco.

1. O que vai na lacuna: beq ou bne?
2. Com x = −5, quanto vale $s0 no final? E com x = 4?
3. Por que aqui não precisou de j?

beq · bne

Atividade

1. **Observar** — montar o aula4_laco.asm e ver os rótulos virarem endereços — 7 min
2. **Prever** — preencher a tabela volta por volta, com um ponto de parada — 10 min
3. **Modificar** — transformar o “maior” em “menor” e prever o resultado — 7 min
4. **Provocar** — mudar n de 8 para 9 — e explicar o número que aparecer — 6 min

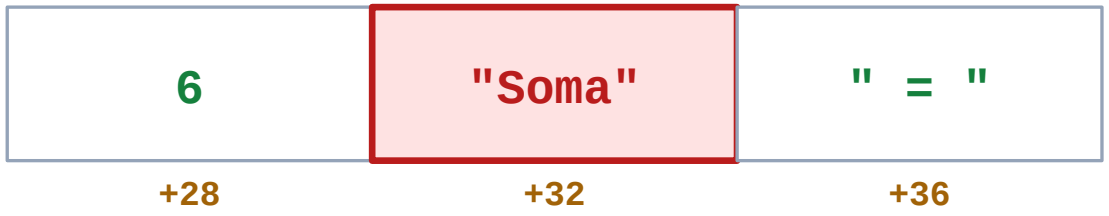
O enunciado está na folha de atividade. Os moldes de if, while e for estão na folha de consulta, na última página.

Uma volta a mais

```
li  $s1, 9
```

```
Run I/O:

Soma = 1634561916
Maior = 1634561875
```



último elemento · 9ª palavra: já é o texto msg1

A 9ª volta leu a palavra depois do vetor — os bytes de "Soma", que valem **`0x616D6F53`** = 1 634 561 875. O processador não sabe onde o vetor acaba.

Quem decide quando parar é o programa. Nenhum erro, nenhum aviso: só um resultado errado. Em C acontece igual — e é uma das falhas de segurança mais antigas que existem.

O que fica de hoje

1. Desviar é escrever outro endereço no PC. Rótulos viram esses endereços.
2. j desvia sempre. beq e bne desviam só se a condição vale.
3. slt compara e responde 1 ou 0. Com beq/bne, vira “menor que”.
4. No if, o desvio usa a condição invertida: ele serve para pular o bloco.
5. Todo for tem quatro partes: inicialização, teste, corpo e incremento.

Com o que vocês já sabem, dá para escrever qualquer programa.

A partir de hoje, vocês já podem começar

Exercício	Enunciado	Quando
`A.6`	Calculadora que acumula inteiros até receber zero	**já**
`A.7`	Soma dos dois maiores entre três valores lidos	**já**
`A.9`	Os cem primeiros números primos, com sub-rotina de primalidade	**depois do encontro 5**
`A.10`	Torres de Hanói, recursivo — para quem quiser desafio	**depois do encontro 5**
`livre`	Conversor entre bases de numeração	**já**

Enunciados: Patterson & Hennessy, Apêndice A, Seção A.12.

Em duplas
as mesmas da atividade, ou não

Assembly MIPS
resultado visível no Run I/O

README.md
o que faz e como executar

Repositório da liga
apresentação no encontro 6

Escolham o enunciado esta semana. Quem começar agora chega ao encontro 5 com uma dúvida de verdade para trazer.

E se o mesmo trecho servir em vários lugares?

```
li   $v0, 1
move $a0, $t1
syscall
…
li   $v0, 1
move $a0, $t2
syscall
```

O programa de hoje imprime dois números com o mesmo trecho, escrito duas vezes. Um j até um trecho comum seria fácil — o difícil é voltar para o lugar certo depois.

`jal` `jr` `$ra`

Um salto que lembra de onde saiu.

Na aula 5: funções — e a pilha.